

Controle de Versão e Configuração

Aula 09 · Bloco 3 — Do Código à Produção

O que mudou, quem mudou — e como voltar ao que funcionava

🎯 Nesta aula

1. O que o **histórico** responde — e a cópia de segurança não
2. **Item de configuração**: o que entra e o que nunca entra
3. **Baseline**: o retrato aprovado, e a mudança rastreável
4. **Estratégias de ramificação**: GitFlow, GitHub Flow e trunk-based development
5. **Versionamento semântico**, release e CHANGELOG

Sábado, 20h: o pagamento parou

Às 19h40, alguém publicou uma correção no frete do **delivery**. Às 20h, os pedidos on-line deixaram de ser confirmados — com o salão cheio.

1

O que mudou?

desde a última vez em que tudo funcionava

2

Quem mudou, e por quê?

a correção do frete tinha mais alguma coisa dentro?

3

Como volto?

agora, sem esperar o conserto

Sem controle de versão, as três se respondem **de memória**. Com ele, cada uma é **um comando**.

O commit: uma fotografia com autor

O Git não guarda "o arquivo mais recente": guarda uma sequência de **commits** — cada um, uma fotografia do projeto inteiro.

identificador	`e81b07d`	*qual versão, exatamente?*
autor e data	`Ana · sáb 24/10, 19:40`	*quem mudou? quando?*
mensagem	`fix: arredonda o frete para cima`	*por que mudou?*
commit anterior	`a3f9c21`	*o que veio antes?*

É o encadeamento pelo **commit anterior** que forma a história.

As três perguntas viram consulta

```
git log --oneline -3      # 1. o que mudou
e81b07d fix: arredonda o frete para cima
a3f9c21 feat: aceita cupom de desconto
7c2d4e0 docs: explica como reabrir a loja

git show e81b07d          # 2. quem, quando, por quê — e cada linha
git revert e81b07d        # 3. volta: um commit novo que desfaz aquele
```

> 💡 **Voltar não apaga a história.** O `revert` cria uma fotografia nova que desfaz a mudança — na segunda-feira, ainda dá para ler o que aconteceu no sábado. **Controle de versão não é backup.**

O Git é uma parte da gerência de configuração

Gerência de versões

que versões de cada peça existem, e como se relacionam?

o Git · esta aula

Construção do sistema

juntar as peças certas num sistema que roda, sempre do mesmo jeito

Aula 10

Gerência de mudanças

quem pede, quem aprova, como se registra cada mudança

esta aula

Gerência de releases

o que vai para quem usa, quando, e com que número

esta aula e a Aula 11

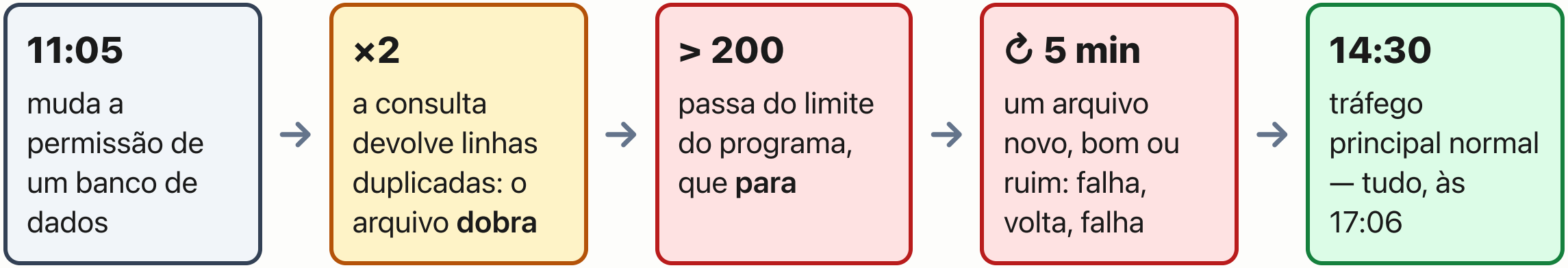
As quatro atividades vêm do Sommerville. **O Git sozinho só faz a primeira.**

🔎 A pergunta-teste

Se isto mudar sozinho,
alguma coisa quebra
ou alguém se engana?

Então é **item de configuração**:
tem versão, tem histórico
e só muda por um caminho combinado.

18/11/2025: o arquivo que não era código



Cloudflare — X e ChatGPT fora do ar por horas. **Nenhuma linha de código mudou.**

> 💡 **Configuração é mudança.** A primeira lição que a empresa listou: tratar a configuração gerada internamente com o mesmo rigor da entrada de usuário — validar antes de distribuir, e distribuir aos poucos.

O que entra, e o que nunca entra

✅ Entra — tem versão

- o código da aplicação
- a configuração por ambiente, **sem segredo**
- os scripts do banco (migrações)
- a lista exata das bibliotecas (*lockfile*)
- o `Dockerfile`, os arquivos do Terraform
- a definição do pipeline
- a documentação de operação

⛔ Nunca entra — e onde fica

- **segredos** → cofre ou variável de ambiente; no repositório, só o `.env.example`
- **bibliotecas baixadas, binários** → gerados de novo a cada construção
- **dados reais** → no banco, com controle de acesso — LGPD
- **arquivos pessoais** → no `.gitignore`

O segredo que foi parar no repositório

23,8 milhões

de segredos expostos em repositórios públicos do GitHub **só em 2024** — 25% a mais que em 2023

70%

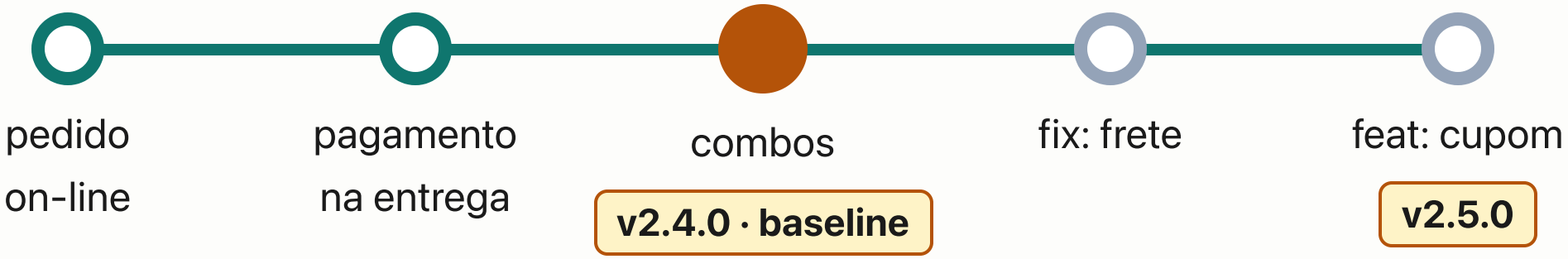
dos segredos vazados em 2022 **continuavam válidos** dois anos depois

Fonte: GitGuardian, *State of Secrets Sprawl 2025*.

⚠️ **Apagar no commit seguinte não resolve:** a história guarda, e toda cópia do repositório também. Primeiro **revogue a credencial** — troque a senha, invalide o token. Só depois limpe o arquivo.

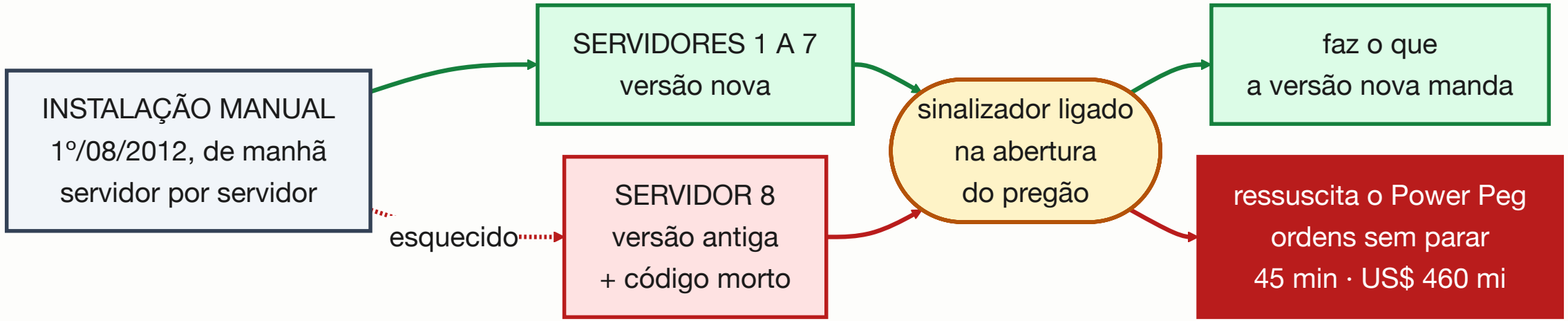
Baseline: o retrato aprovado

Itens de configuração em versões específicas, **revisados e aprovados**, que passam a servir de referência. No Git, ela se marca com uma **tag** — um nome fixo preso a um commit.



É a linha de base da Aula 03, aplicada ao produto. A `v2.4.0` não se move, por regra do time — e responde *o que, exatamente, está no ar?*

1º/08/2012: oito servidores, sete atualizados



Knight Capital. O defeito não estava no código novo: **ninguém sabia, com certeza, o que rodava em cada servidor** — e o código morto nunca tinha saído.

As quatro funções — no prontuário de clínica-escola

Identificação

quais são os itens? código, migrações, a regra de quem vê qual prontuário, o termo de consentimento

Controle de mudanças

quem muda o quê, por qual caminho? a regra de acesso só muda com revisão de duas pessoas

Contabilização da situação

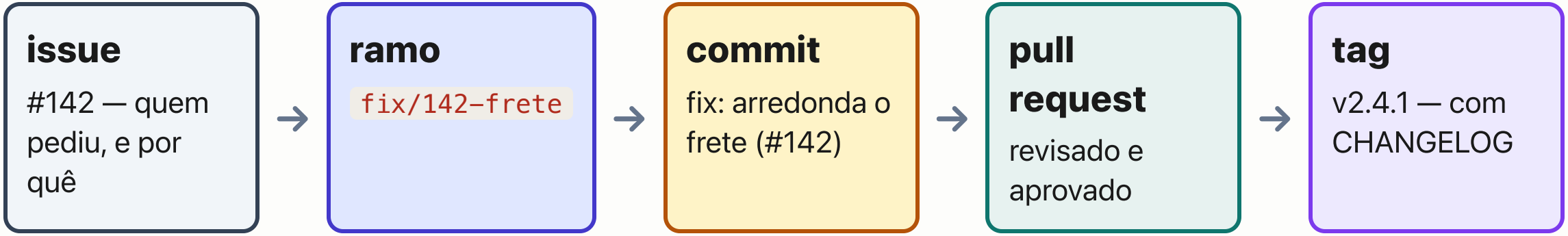
o que está onde, agora? *"produção roda a v3.2.1 desde 14/10"*

⚠️ foi aqui que a Knight falhou

Auditoria de configuração

o que está no ar é o que foi aprovado? a auditoria externa confere todo ano

Da solicitação à release: a cadeia rastreável



O **pull request** é o pedido para integrar um ramo: mostra cada linha que muda e é onde a mudança é revisada. É **controle de mudança e registro** ao mesmo tempo.

"Por que o frete mudou na 2.4.1?" — trinta segundos. É exatamente o que a auditoria do prontuário pede.

Nem toda mudança pede o mesmo caminho

Padrão

tirar do cardápio o item que acabou

pré-aprovada: segue o roteiro e registra

Normal

mudar a regra de cálculo do frete

impacto, revisão, aprovação, nova baseline

Emergencial

o pagamento parou às 20h de sábado

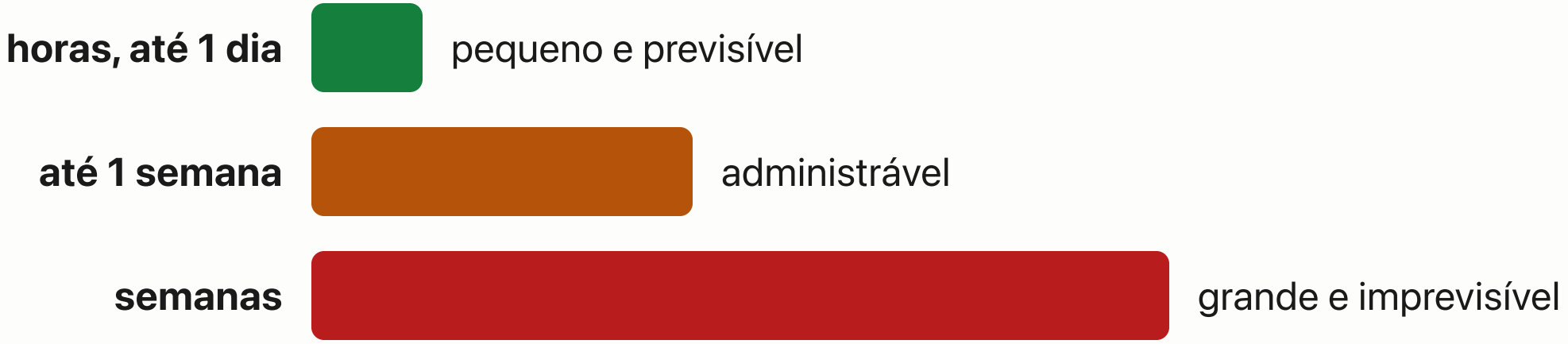
executa já e **registra depois, com prazo**

Os três tipos são do ITIL.

⚠️ **Processo pesado demais é contornado, não seguido.** E o emergencial precisa dizer o que **autoriza pular** — a aprovação prévia, não a revisão — e o que **obriga a fazer depois**.

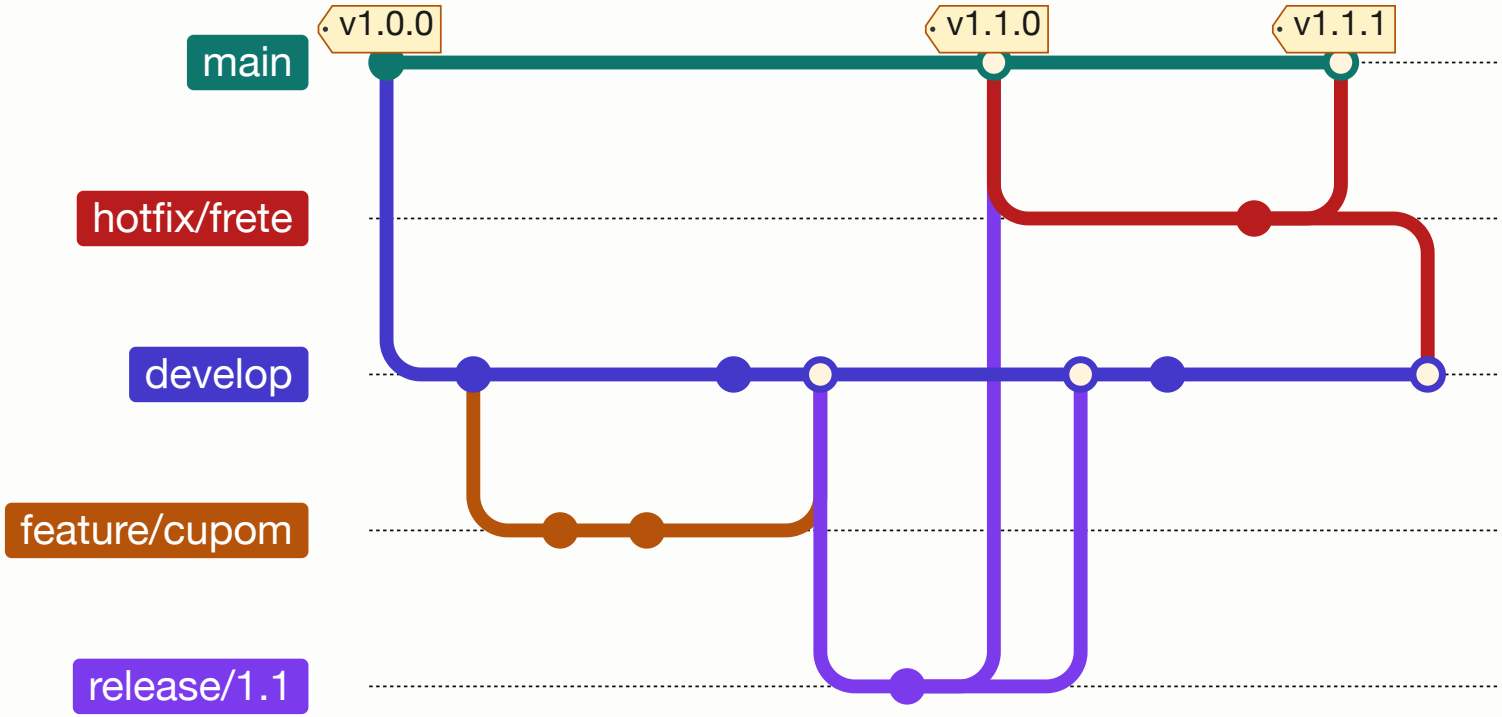
O custo do ramo que vive demais

Ana integra o ramo dela no dia seguinte. Bruno integra o dele três semanas depois — e, nesse tempo, o principal recebeu trinta outras mudanças.



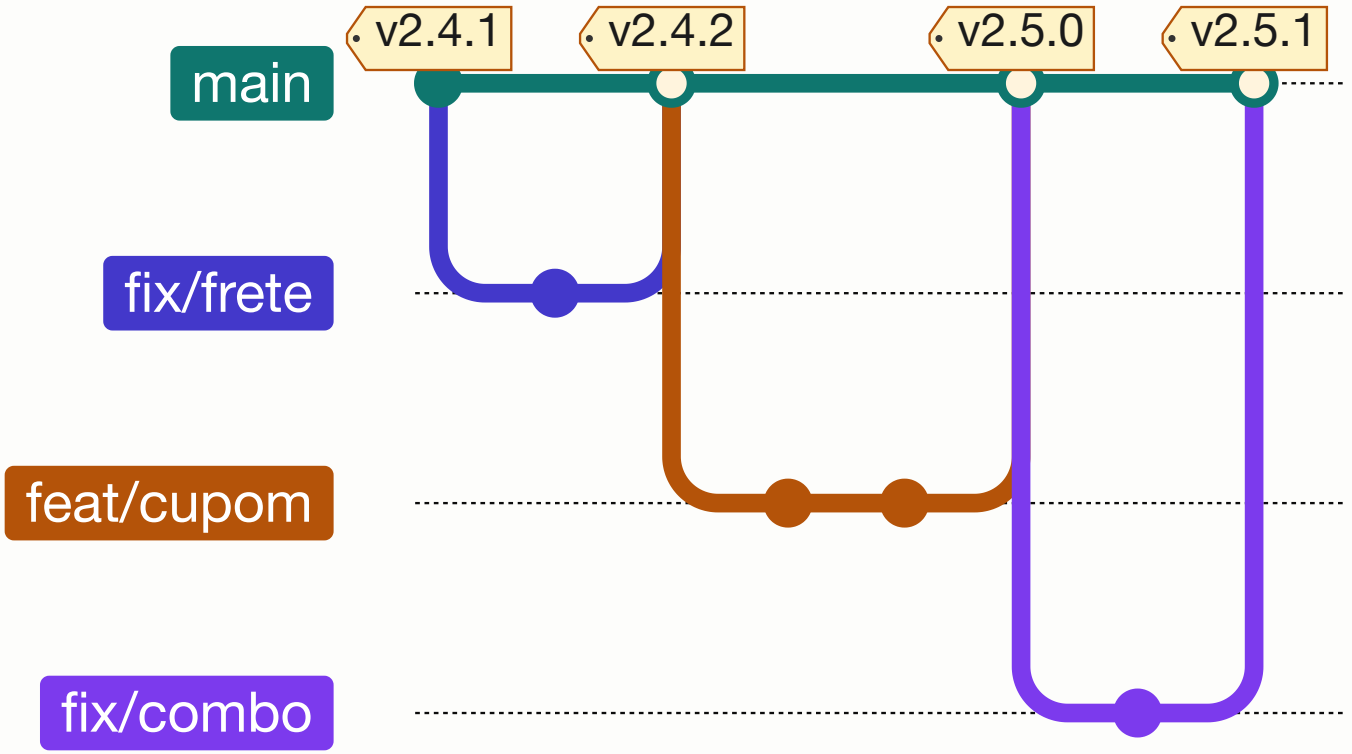
Adiar a integração não evita o custo: multiplica. É a curva de custo da Aula 02 — e o trabalho parcialmente feito que o Lean chama de desperdício.

GitFlow — várias versões em uso ao mesmo tempo



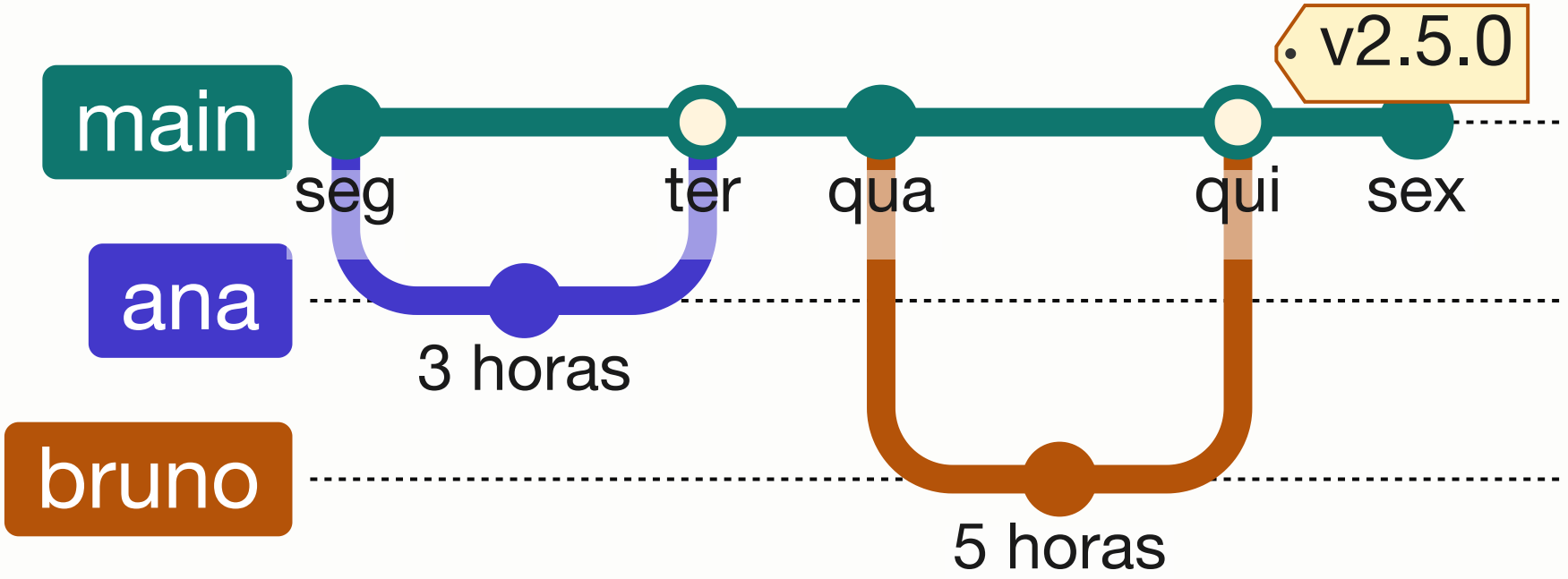
Para aplicativo instalado, biblioteca, sistema vendido a clientes. **O próprio autor, em 2020:** para aplicação web entregue continuamente, use algo mais simples.

GitHub Flow — uma versão em produção



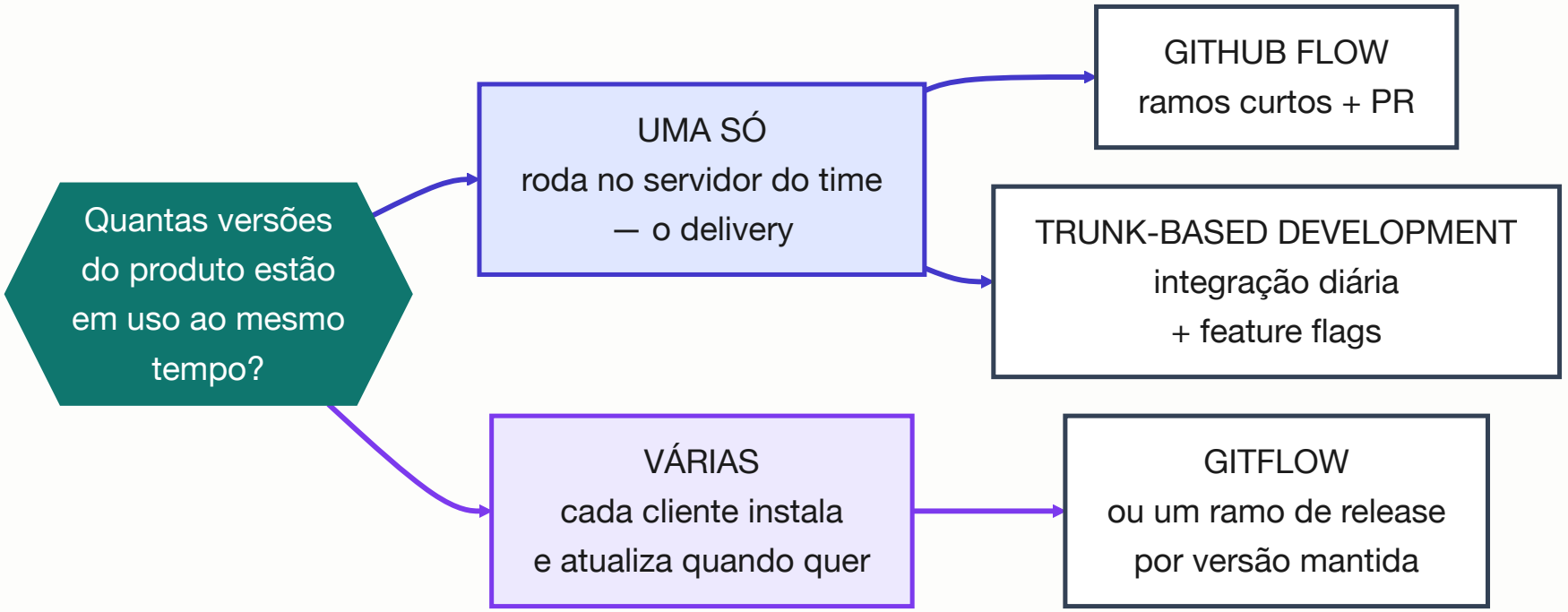
Um ramo permanente, **sempre pronto para ir ao ar**. Todo trabalho num ramo curto, que termina num pull request revisado.

Trunk-based development — todo mundo integra, todo dia



Ramos de horas. O que ainda não está pronto entra **desligado**, atrás de uma *feature flag* — Aula 11.

Como escolher



Aprovação formal não exige ramo longo. O comitê do prontuário aprova uma *tag* — e o time continua integrando no `main`.

O que a pesquisa mostra

≤ 3

ramos ativos no repositório

1× / dia

integração no `main`, no mínimo

0

períodos de congelamento de código

Times assim entregam com mais velocidade **e** mais estabilidade — DORA, o programa de pesquisa sobre entrega de software mantido pelo Google.

> ⚠️ **Ramo longo não é cuidado: é risco adiado.** O trabalho parece pronto no quadro e ainda não passou pelo único teste que importa — funcionar junto com o resto.

Versionamento semântico

2.4.1

MAIOR

a mudança **quebra** quem depende do sistema

MENOR

funcionalidade nova, **sem quebrar** nada

CORREÇÃO

corrige defeito, **sem quebrar** nada

Ao subir uma parte, as da direita voltam a zero: `2.4.1 → 2.5.0`. Antes do `1.0.0`, tudo pode mudar.

Qual é a próxima versão, a partir da 2.4.1?

Mudança	Quebra quem usa?	Próxima
o frete arredondava errado	não	**2.4.2**
aceita cupom, num campo opcional	não	**2.5.0**
o campo `endereco` vira `enderecoEntrega`	**sim**	**3.0.0**
o CPF passa a ser obrigatório no pedido	**sim**	**3.0.0**

⚠️ **O número é uma promessa.** Quebrar numa correção ensina todo mundo a travar a versão — e a deixar de receber até as correções de segurança.

A mensagem do commit decide a versão

```
fix: arredonda o frete para cima
```
→ **CORREÇÃO** 2.4.2

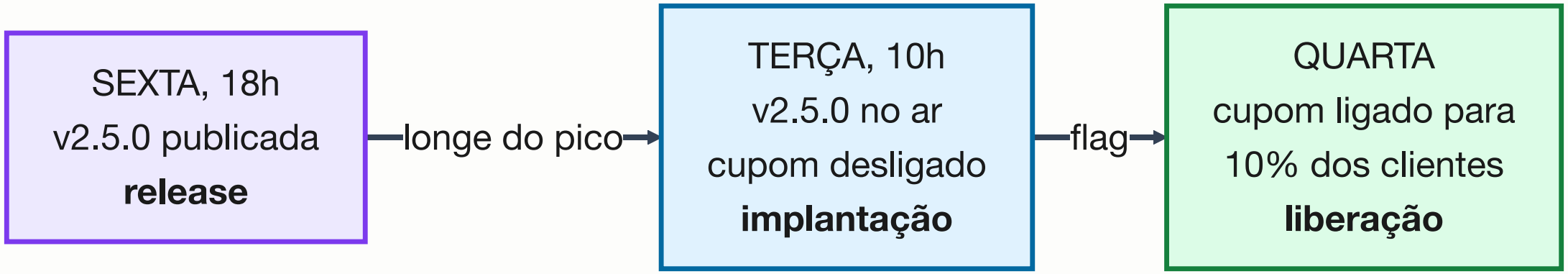
```
feat: aceita cupom de desconto
```
→ **MENOR** 2.5.0

```
feat!: renomeia endereco
```
→ **MAIOR** 3.0.0

```
docs: explica como reabrir a loja
```
→ **não muda**

Conventional Commits. Ferramentas como o *semantic-release* leem o histórico e calculam o número — mas **quem diz se quebra é quem escreve o commit**.

Publicar não é colocar no ar



A **release** é a baseline publicada: tag, notas e pacotes. Pôr no ar e ligar para os clientes são outras duas decisões — e a Aula 11 é sobre separá-las.

O CHANGELOG é para pessoas

[3.0.0] — 2026-10-29

Modificado **Quebra compatibilidade:** `endereco` passou a se chamar `enderecoEntrega`

Removido pagamento por boleto, sem uso desde agosto

[2.5.0] — 2026-10-22

Adicionado cupom de desconto no pedido

[2.4.2] — 2026-10-18

Corrigido o frete era arredondado para baixo acima de 8 km

Segurança atualizada a biblioteca de pagamento, que tinha uma falha conhecida

Keep a Changelog: a versão mais recente primeiro, data no formato ISO, seis tipos de mudança. Falta um aqui: **Obsoleto** — o boleto merecia uma versão de aviso antes de sair.

Log de commits não é CHANGELOG

```
e81b07d ajusta teste do carrinho
9f1c2aa Merge branch 'fix/frete'
4b7e0d1 corrige typo
c0ffee1 wip
5d3a9b2 feat!: renomeia endereco
8a2b4c6 atualiza dependências
77d0e3f ajusta lint
2e9f1b0 Merge branch 'feat/cupom'
```

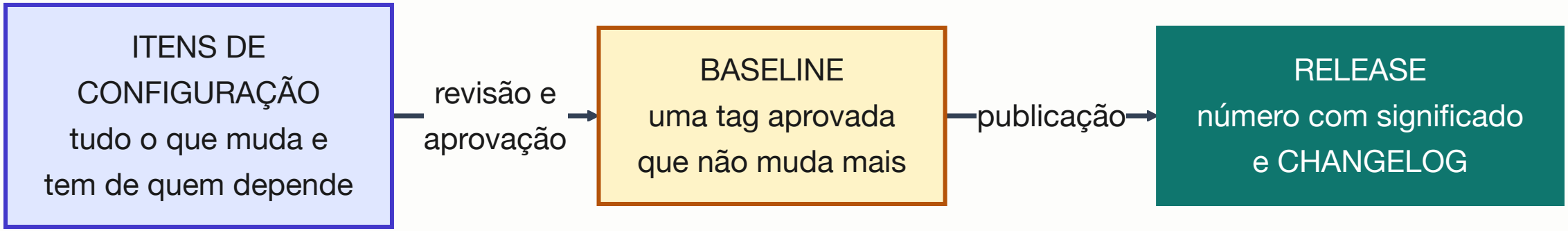
O que o dono do restaurante precisava ler

uma linha só: **a 3.0.0 quebra o aplicativo do entregador**, que precisa estar na versão 5.0.

No log, essa linha está no meio de 47 outras.

O log é para quem **desenvolve**. O CHANGELOG, para quem **usa e opera**.

O que esta aula deixa no lugar



Falta garantir que **cada** mudança integrada ao `main` não quebrou nada. Conferir à mão, a cada commit, não escala.

➡️ Próxima aula

Aula 10 — Integração contínua

Integrar todo dia só é seguro
se uma máquina conferir
cada integração.

É o que o pipeline faz.